UPTIME

Plataforma de Observabilidad y Monitorización de Infraestructura en Tiempo Real

Alumno: **Jaime**

Centro: **CESUR FP — Campus Cartuja, Sevilla**

Ciclo: **Grado Superior en Desarrollo de Aplicaciones Web (DAW)**

Tutor: **David Postigo Rodríguez**

Curso Académico: **2024 – 2025**

Convocatoria: **2.º Trimestre · Proyecto Integrado**

ÍNDICE DE CONTENIDOS

Resumen Ejecutivo **3**

01 Introducción y Análisis Inicial **4**

1.1 Descripción del Proyecto 4
1.2 Justificación 4
1.3 Análisis de Requisitos 4

02 Objetivos del Proyecto **5**

2.1 Objetivos Funcionales 5
2.2 Objetivos No Funcionales 5

03 Historias de Usuario **6**

04 Casos de Uso **7**

CU-01 Vinculación de un nuevo servidor 7
CU-02 Notificación automática de incidencia 7
CU-03 Consulta del historial de rendimiento 8

05 Diseño del Sistema **9**

5.1 Arquitectura Técnica 9
5.2 Stack Tecnológico 9
5.3 Esquema Entidad-Relación 9
5.4 Relaciones entre Entidades 10

06 Wireframes y Mockups **11**

Pantalla 1 — Dashboard Global 11
Pantalla 2 — Inventario de Servidores 11
Pantalla 3 — Gestión de Alertas 12
Pantalla 4 — Registro de Logs 12

07 Estado del Proyecto y Próximos Pasos **13**

7.1 Estado Actual 13
7.2 Metodología y Organización 13

Bibliografía y Referencias **14**

RESUMEN EJECUTIVO

Uptime centraliza la monitorización de servidores en tiempo real. Recibe métricas de CPU, RAM y almacenamiento desde agentes externos, las visualiza en un dashboard reactivo mediante WebSockets y dispara alertas automatizadas al superar umbrales críticos.

Tecnología	Rol
Laravel 11 · PHP 8.3	Backend MVC, ORM, colas y observers
Livewire 3	Componentes reactivos sin JavaScript manual
Laravel Reverb	Servidor WebSocket oficial — eventos en tiempo real
Tailwind CSS v3	Diseño responsivo Mobile First
MySQL 8.0	Persistencia de métricas, alertas y logs
Laravel Sanctum	Tokens de API y sesión del administrador

01 INTRODUCCIÓN Y ANÁLISIS INICIAL

1.1 Descripción del Proyecto

Uptime es una plataforma web de observabilidad y monitorización de infraestructura en tiempo real diseñada para desarrolladores y administradores de sistemas. La aplicación recibe flujos continuos de métricas —carga de CPU, consumo de RAM y espacio en disco— provenientes de agentes ligeros instalados en cada servidor y las refleja al instante en gráficos reactivos sin necesidad de recargar la página.

1.2 Justificación

Las soluciones de observabilidad del mercado —Datadog, New Relic, Grafana Cloud— presentan barreras de entrada significativas: costes elevados por nodo, dependencia de infraestructura en la nube de terceros y curvas de configuración pronunciadas. Uptime nace como alternativa autohospedada, ligera y accesible, construida sobre el ecosistema PHP/Laravel, lo que permite desplegarla y mantenerla sin coste de licencias ni dependencias externas.

1.3 Análisis de Requisitos

- **Recepción en tiempo real:** El sistema procesa flujos continuos de métricas enviadas por agentes externos instalados en los servidores.
- **Interfaz reactiva sin recarga:** La UI se actualiza automáticamente al recibir nuevas métricas mediante WebSockets.
- **Persistencia histórica:** Todas las métricas se almacenan en base de datos para análisis retrospectivo en cualquier rango temporal.
- **Alertas configurables:** El administrador define umbrales críticos por servidor y métrica y recibe notificaciones automáticas.
- **Seguridad por diseño:** Cada servidor se autentica con token único; el panel está protegido por autenticación robusta.
- **Responsividad:** La interfaz se adapta correctamente a dispositivos móviles y de escritorio.

02 OBJETIVOS DEL PROYECTO

2.1 Objetivos Funcionales

ID	Objetivo	Descripción
OF-01	Gestión de servidores	CRUD completo de nodos vinculados al perfil: alta, edición, eliminación y estado en tiempo real.
OF-02	Monitorización en tiempo real	Visualización dinámica de CPU, RAM y disco mediante WebSockets (Laravel Reverb), sin recargar la página.
OF-03	Sistema de alertas	Configuración de umbrales críticos por métrica. Al superarlos se envía notificación vía Email o Telegram.
OF-04	Panel de logs centralizado	Registro y visualización de eventos por servidor para facilitar el diagnóstico remoto de incidencias.
OF-05	Autenticación y API tokens	Registro de usuarios, sesión segura y generación de tokens únicos por servidor para los agentes.

2.2 Objetivos No Funcionales

Dimensión	Descripción
Rendimiento	El backend procesará métricas de múltiples agentes de forma asíncrona mediante colas (Laravel Queues), evitando bloqueos en el hilo HTTP principal.
Seguridad	Autenticación robusta con Laravel Sanctum. Tokens únicos por servidor. Validación exhaustiva de toda entrada de datos.
Usabilidad	Interfaz Mobile First con Tailwind CSS. La monitorización debe ser posible desde cualquier dispositivo sin pérdida de funcionalidad.
Escalabilidad	Arquitectura desacoplada para añadir nuevos tipos de métricas o canales de notificación sin refactorizaciones profundas.
Mantenibilidad	Código estructurado según el patrón MVC de Laravel, con comentarios y separación de responsabilidades.

03 HISTORIAS DE USUARIO

Uptime tiene un único tipo de usuario autenticado: el Administrador, responsable de gestionar los servidores, configurar alertas y analizar métricas. Toda interacción con el sistema pasa por este rol.

ID	Historia de Usuario
HU-01	Como administrador, quiero registrarme en la plataforma y obtener una clave de API única para poder conectar mis servidores de forma segura.
HU-02	Como administrador, quiero ver gráficos actualizados en tiempo real con las métricas de CPU, RAM y disco para detectar problemas en el momento exacto en que ocurren.
HU-03	Como administrador, quiero configurar una regla de alerta que me notifique cuando la RAM supere el 85% para poder actuar antes de que el servicio se vea comprometido.
HU-04	Como administrador, quiero añadir, editar y eliminar servidores para mantener actualizado el inventario de activos que gestiono.
HU-05	Como administrador, quiero consultar el historial de métricas filtrando por rango de fechas para analizar tendencias y planificar ampliaciones de capacidad.
HU-06	Como administrador, quiero ver un panel centralizado de logs para diagnosticar incidencias de forma remota sin acceder directamente al servidor.

04 CASOS DE USO

CU-01 — Vinculación de un nuevo servidor

Campo	Detalle
Actor	Administrador
Precondición	El usuario tiene sesión activa en la plataforma.
Flujo principal	1. El administrador pulsa «Añadir servidor». 2. El sistema genera un identificador único y un token secreto. 3. El administrador instala el agente con las credenciales generadas. 4. El agente comienza a enviar tramas de métricas al endpoint de la API. 5. El sistema valida el token y actualiza el estado del nodo a «Activo». 6. El dashboard refleja el nuevo servidor con sus métricas en tiempo real.
Postcondición	El servidor queda registrado como activo y sus métricas son visibles desde el panel.

CU-02 — Notificación automática de incidencia

Campo	Detalle
Actor	Sistema (disparado automáticamente)
Precondición	Existe al menos una regla de alerta activa para el servidor afectado.
Flujo principal	1. El agente envía una métrica que supera el umbral (ej. CPU > 90%). 2. El Observer de Laravel detecta la anomalía al persistir la métrica. 3. Se encola una tarea de notificación (Laravel Queue). 4. El worker procesa la cola y envía la alerta por Email / Telegram. 5. El incidente queda registrado con marca de tiempo y valor exacto. 6. El administrador recibe la notificación y puede consultar el contexto.
Postcondición	El administrador es notificado y el evento queda registrado para auditoría.

CU-03 — Consulta del historial de rendimiento

Campo	Detalle
Actor	Administrador
Precondición	El servidor existe y tiene métricas almacenadas en base de datos.
Flujo principal	1. El administrador selecciona un servidor y accede a su vista detallada. 2. Elige un rango de fechas mediante el selector temporal del panel. 3. El sistema consulta la tabla metrics filtrando por server_id e intervalo. 4. Se renderizan gráficos de área con el histórico de CPU, RAM y disco. 5. El administrador correlaciona picos de uso con los logs del mismo período.
Postcondición	El administrador visualiza el histórico completo en el rango seleccionado.

05 DISEÑO DEL SISTEMA

5.1 Arquitectura Técnica

Uptime sigue el patrón MVC de Laravel 11, extendido con Livewire 3 para dotar a las vistas Blade de reactividad sin JavaScript manual. La comunicación en tiempo real entre el backend y el navegador se gestiona mediante Laravel Reverb, el servidor WebSocket oficial del ecosistema Laravel.

Agente Externo	→	Laravel API	→	Reverb (WS)	→	Dashboard Reactivo
HTTP POST /api/metrics		Controller + Observer		WebSocket Server		Livewire Component

5.2 Stack Tecnológico

Capa	Tecnología	Versión	Rol en el proyecto
Backend	Laravel 11 · PHP 8.3	11 / 8.3	Framework MVC. Rutas, controladores, Eloquent ORM, colas y observers.
Frontend	Livewire 3	Última	Componentes reactivos que conviven con Blade sin framework JS adicional.
WebSockets	Laravel Reverb	Última	Servidor WebSocket oficial de Laravel. Emite eventos de métricas en tiempo real.
Estilos	Tailwind CSS	v3	Framework CSS utility-first. Diseño responsivo con enfoque Mobile First.
Base de Datos	MySQL	v8.0	Persistencia de usuarios, servidores, métricas, alertas y logs.
Autenticación	Laravel Sanctum	Última	Tokens de API por servidor y gestión de sesión web del administrador.
Colas	Laravel Queues	Built-in	Procesamiento asíncrono del envío de notificaciones.
Versiones	Git + GitHub	—	Gestión del código fuente, tablero Kanban y seguimiento del proyecto.

5.3 Esquema Entidad-Relación

Tabla	Campos principales	Descripción
users	id · name · email · password · created_at	Almacena los administradores. Cada usuario accede solo a sus propios servidores.
servers	id · user_id (FK) · name · ip_address · api_token · status	Cada servidor monitorizado. api_token autentica al agente. Status: activo / inactivo / error.

Tabla	Campos principales	Descripción
metrics	id · server_id (FK) · cpu_load · ram_usage · disk_free · created_at	Cada trama de métricas recibida. Base para gráficos en tiempo real e historial.
alert_configs	id · server_id (FK) · metric_type · threshold_value · channel · active	Configuración de umbrales por servidor y métrica. Campo channel: email o telegram.
alert_logs	id · alert_config_id (FK) · triggered_at · value_at_trigger	Historial de alertas disparadas con el valor exacto en el momento de la incidencia.

5.4 Relaciones entre Entidades

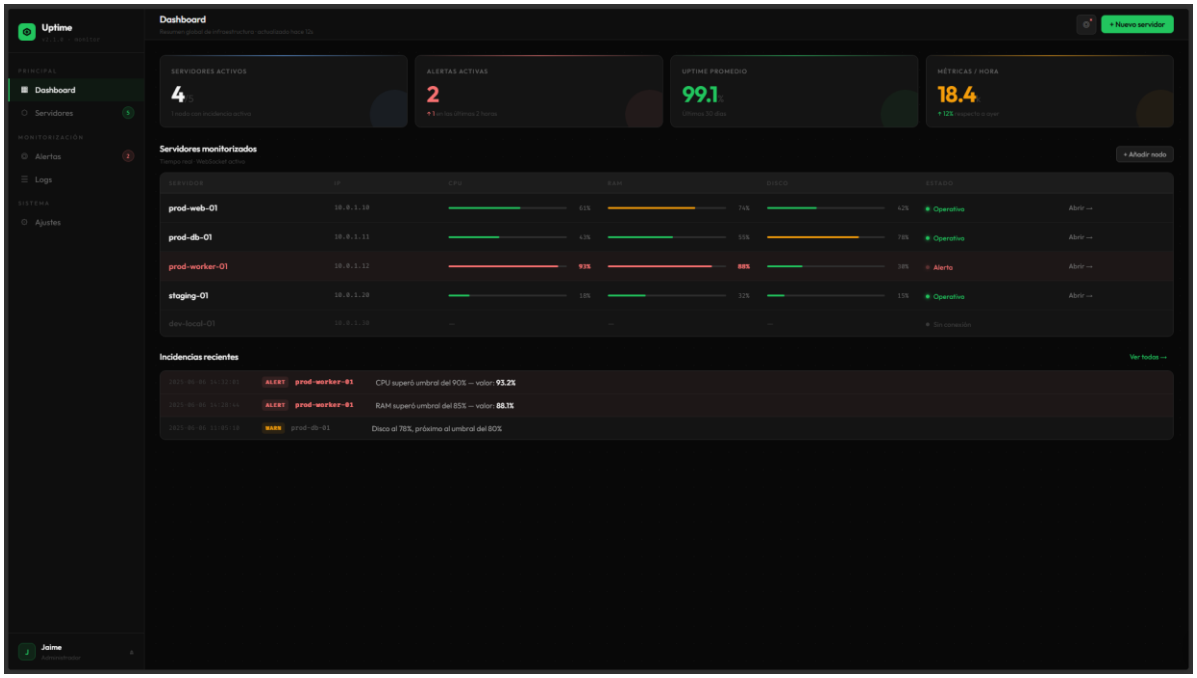
- **User → Servers:** Un usuario puede gestionar múltiples servidores (relación 1:N).
- **Server → Metrics:** Un servidor genera múltiples registros de métricas a lo largo del tiempo (1:N).
- **Server → Alert Configs:** Un servidor puede tener varias configuraciones de alerta, una por métrica vigilada (1:N).
- **Alert Config → Alert Logs:** Cada configuración genera un log cada vez que se activa (1:N).

06 WIREFRAMES Y MOCKUPS

Las siguientes capturas corresponden a la implementación real de la interfaz de Uptime, siguiendo un estilo dark dashboard coherente con la naturaleza técnica de la herramienta. El diseño está construido con Tailwind CSS bajo un enfoque Mobile First.

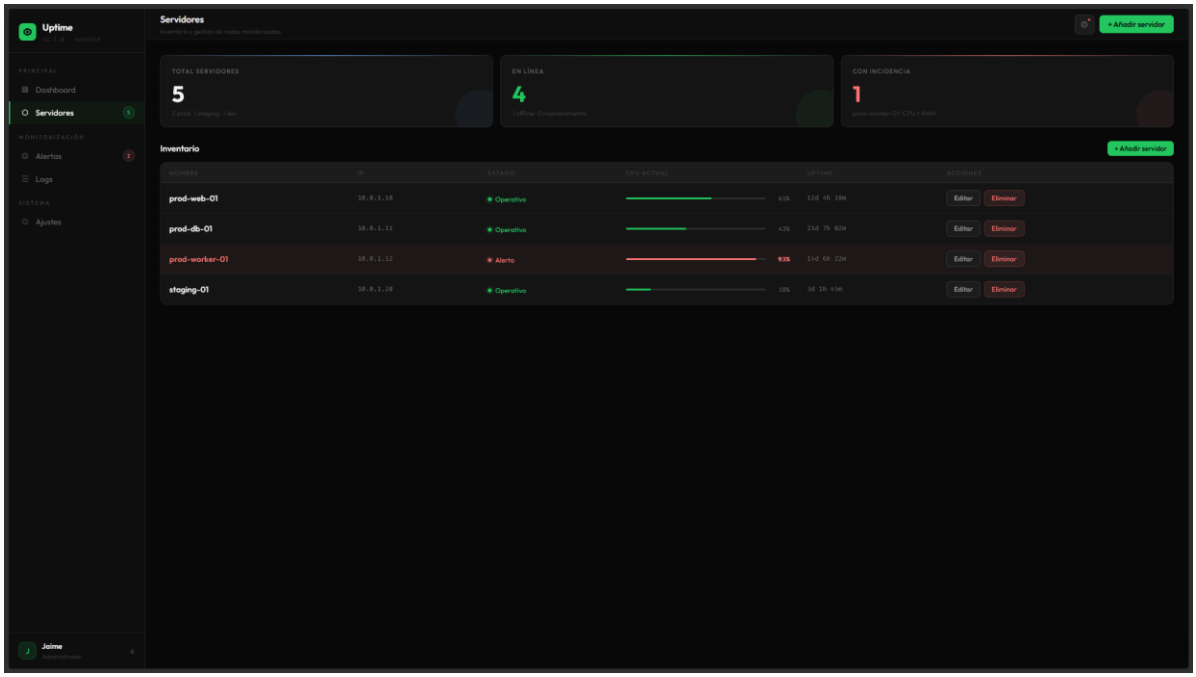
Pantalla 1 — Dashboard Global

Panel principal con KPIs globales y tabla de servidores monitorizados. Las métricas se actualizan en tiempo real mediante WebSockets sin recargar la página. Se muestra el estado de cada nodo con barras de progreso para CPU, RAM y disco.



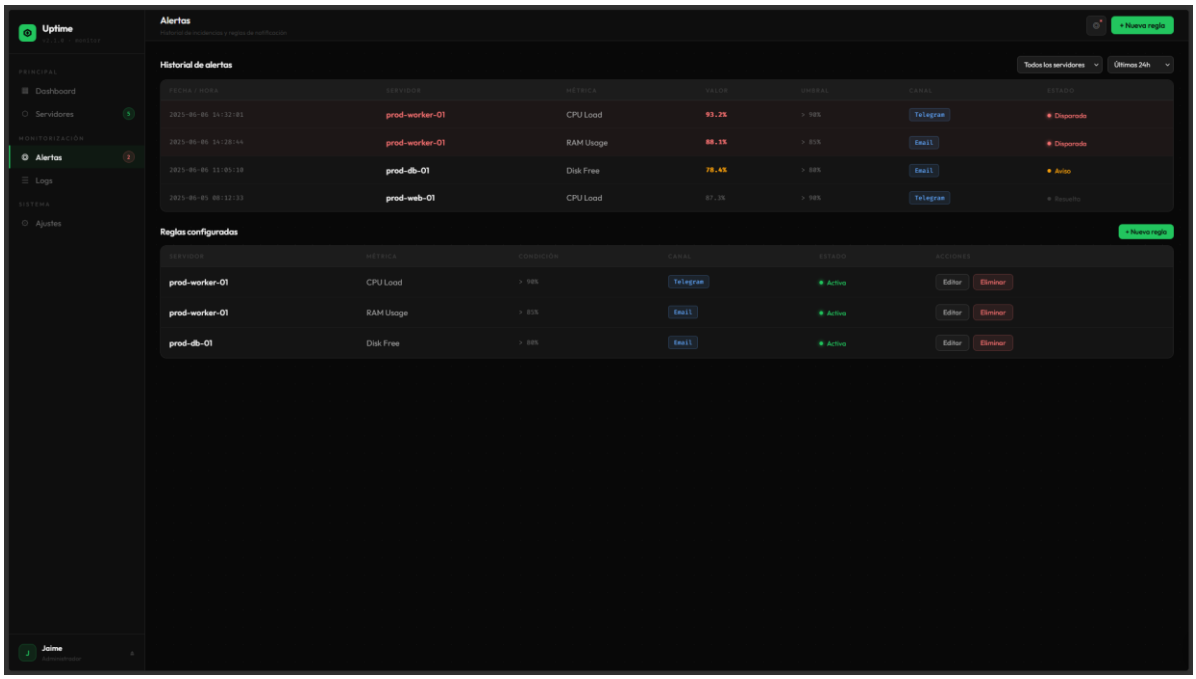
Pantalla 2 — Inventario de Servidores

Vista de inventario con 3 KPIs superiores (total, en línea, con incidencia) y tabla completa con estado, CPU actual y uptime de cada nodo. Incluye acciones de editar y eliminar por fila.



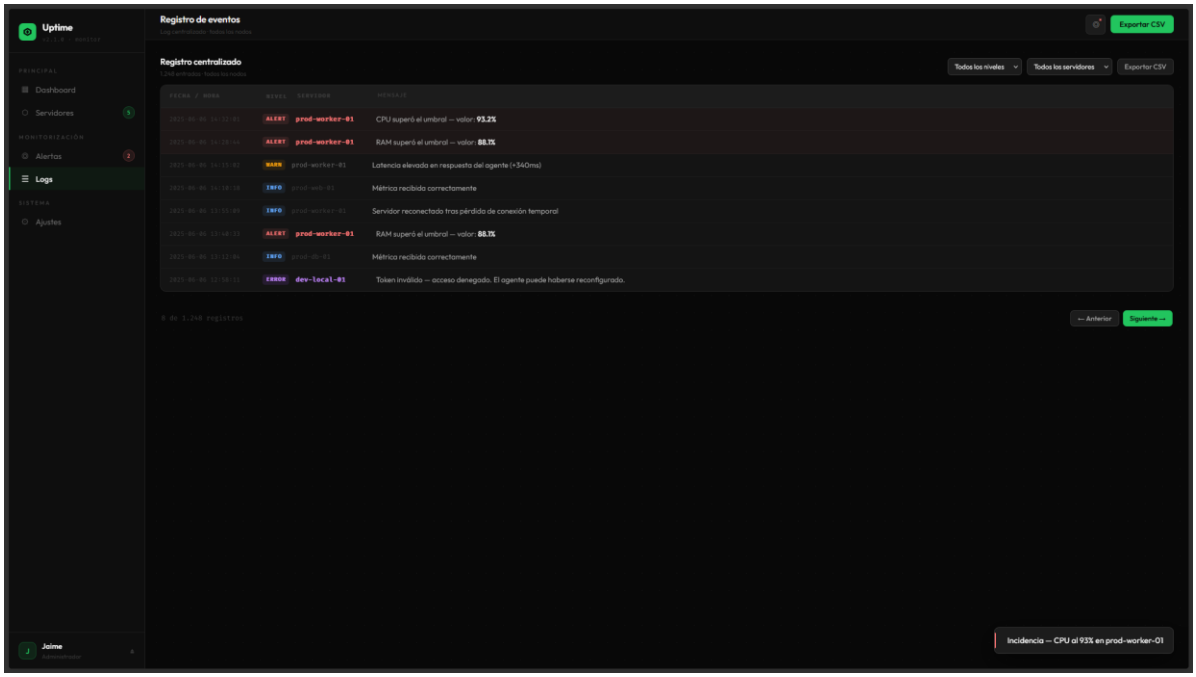
Pantalla 3 — Gestión de Alertas

Historial de alertas disparadas (panel superior) con fecha, servidor, métrica, valor, umbral, canal y estado. Panel inferior con las reglas configuradas por servidor. Filtros por servidor y rango temporal.



Pantalla 4 — Registro Centralizado de Logs

Log centralizado con todos los eventos de infraestructura. Filtros por nivel (INFO / WARN / ALERT / ERROR) y por servidor. Paginación y exportación a CSV. Toast de notificación en tiempo real en la esquina inferior.


Uptime
Registro de eventos
Exportar CSV
Registro centralizado
Todos los niveles
Todos los servidores
Exportar CSV
PRINCIPAL
Dashboard
Servidores
MONITORIZACIÓN
Alertas
Logs
SISTEMA
Ajustes
FECHA / HORA
NIVEL
SERVIDOR
MENSAJE
ALERT prod-worker-01 CPU superó el umbral — valor: 93.2%
ALERT prod-worker-01 RAM superó el umbral — valor: 88.7%
WARN prod-worker-01 Latencia elevada en respuesta del agente (+340ms)
INFO prod-web-01 Métrica recibida correctamente
INFO prod-worker-01 Servidor reconectado tras pérdida de conexión temporal
ALERT prod-worker-01 RAM superó el umbral — valor: 88.7%
INFO prod-db-01 Métrica recibida correctamente
ERROR dev-local-01 Token inválido — acceso denegado. El agente puede haberse reconfigurado.
8 de 1.248 registros
← Anterior
Siguiente →
Jaime
Administrador
Incidencia — CPU al 93% en prod-worker-01

07 ESTADO DEL PROYECTO Y PRÓXIMOS PASOS

6.1 Estado Actual

Fase	Tarea	Estado
Análisis	Definición de requisitos y casos de uso	✓Completado
Análisis	Historias de usuario	✓Completado
Diseño	Selección del stack tecnológico	✓Completado
Diseño	Modelo de datos — ERD	✓Completado
Diseño	Wireframes de las pantallas principales	✓Completado
Desarrollo	Instalación y configuración de Laravel 11	◯En progreso
Desarrollo	Migraciones, modelos y seeders	◯En progreso
Desarrollo	Autenticación y API tokens (Sanctum)	○ Pendiente
Desarrollo	Integración de Laravel Reverb — WebSockets	○ Pendiente
Desarrollo	Componentes Livewire — Dashboard reactivo	○ Pendiente
Desarrollo	Sistema de alertas y colas (Queues + Notifications)	○ Pendiente
Desarrollo	Panel de logs centralizado	○ Pendiente
Pruebas	Tests unitarios y funcionales (Pest)	○ Pendiente
Entrega	Despliegue en servidor y documentación final	○ Pendiente

6.2 Metodología y Organización

El proyecto se gestiona mediante Scrum adaptado, con sprints semanales definidos al inicio de cada semana y revisados al final. El seguimiento se realiza a través del tablero Kanban de GitHub Projects con las columnas To Do, In Progress y Done.

El código se publica en un repositorio de GitHub con commits frecuentes y mensajes descriptivos siguiendo la convención Conventional Commits. Cada funcionalidad se desarrolla en una rama independiente y se integra en main una vez validada.

BIBLIOGRAFÍA Y REFERENCIAS

- [1] Laravel LLC (2024). Laravel 11.x Documentation. https://laravel.com/docs/11.x
- [2] Livewire (2024). Livewire 3.x — Full-stack framework for Laravel. https://livewire.laravel.com
- [3] Laravel (2024). Laravel Reverb — Real-time WebSockets. https://reverb.laravel.com
- [4] Tailwind Labs (2024). Tailwind CSS — Utility-First CSS Framework. https://tailwindcss.com
- [5] Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.